

Asesor de Negocios IA

Bot de Discord — Sistema Experto

Cafetería de Especialidad en la Zona Metropolitana de Guadalajara

Manual de Usuario y Documentación Técnica

Proyecto SE

14 de junio de 2026

Índice

1. Introducción	3
1.1. Objetivo pedagógico	3
1.2. Tecnologías empleadas	3
2. Arquitectura del Sistema	3
2.1. Diagrama de agentes	3
2.2. Descripción de los agentes	4
2.3. Flujo completo de un ciclo	5
3. Motor de Inferencias	5
3.1. Arquitectura del motor	5
3.2. Catálogo de reglas	6
3.3. Estructura de datos Inferencia	6
4. Modelo Económico	7
4.1. Fórmulas principales	7
4.1.1. Demanda con elasticidad-precio	7
4.1.2. Unidades vendidas	7
4.1.3. Margen de contribución	7
4.1.4. Punto de equilibrio	7
4.1.5. Flujo neto	7
4.1.6. Costo variable unitario con merma	7
4.2. Parámetros calibrados a GDL (MXN)	8
4.2.1. Ejemplo: costos variables calculados	8
4.3. Sistema de zonas — ZMG	8
4.3.1. Análisis de rentabilidad por zona	9

5. Base de Datos	10
5.1. Diagrama entidad-relación	10
5.2. Descripción de tablas	10
6. Eventos Macroeconómicos	11
7. Sistema de Inventario	12
7.1. Insumos disponibles	12
7.2. Flujo del inventario	12
8. Sistema de Marketing	12
9. Referencia de Comandos	13
9.1. Componentes interactivos (botones y selectores)	13
10. Guía de Inicio Rápido	13
11. Ejemplo de Sesión	14
12. Consideraciones Técnicas	15
12.1. Persistencia y transacciones	15
12.2. Pureza del motor de reglas	15
12.3. Escalabilidad del catálogo de reglas	15
12.4. Tests sugeridos (pytest)	15

1. Introducción

Asesor de Negocios IA es un simulador económico de una cafetería de especialidad ubicada en la Zona Metropolitana de Guadalajara (ZMG), implementado como bot de Discord. El sistema constituye un **sistema experto**: el dominio de conocimiento es la gestión económica del negocio, y el simulador aplica ese conocimiento a través de reglas de inferencia para guiar al usuario en sus decisiones.

1.1. Objetivo pedagógico

El proyecto demuestra los componentes fundamentales de los sistemas expertos:

- **Base de conocimiento:** fórmulas económicas y reglas IF-THEN calibradas a la realidad de GDL (precios MXN, zonas, eventos locales).
- **Motor de inferencia:** evaluación de reglas sobre el contexto del negocio.
- **Interfaz de explicación:** el Agente Supervisor traduce las decisiones del sistema a lenguaje natural.

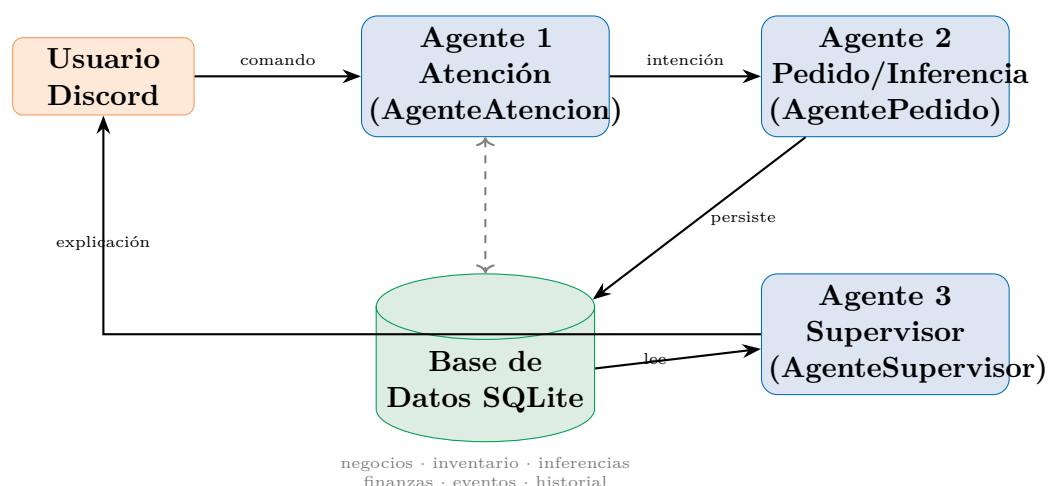
1.2. Tecnologías empleadas

Componente	Tecnología
Plataforma	Discord (slash commands)
Lenguaje	Python 3.11+
Framework	discord.py 2.x
Base de datos	SQLite 3
ORM	Sin ORM (sqlite3 nativo)

2. Arquitectura del Sistema

El sistema sigue un esquema de **tres agentes** como requisito de la rúbrica. Cada agente tiene una responsabilidad única y se comunica con los demás a través de la base de datos compartida.

2.1. Diagrama de agentes



2.2. Descripción de los agentes

Agente 1 — Atención (AgenteAtencion)

Interpreta el comando o texto del usuario y lo convierte en una **Intención estructurada** (accion, parametros). Acciones posibles: avanzar_ciclo, comprar, ajustar_precio, desconocida.

Agente 2 — Pedido/Inferencia (AgentePedido)

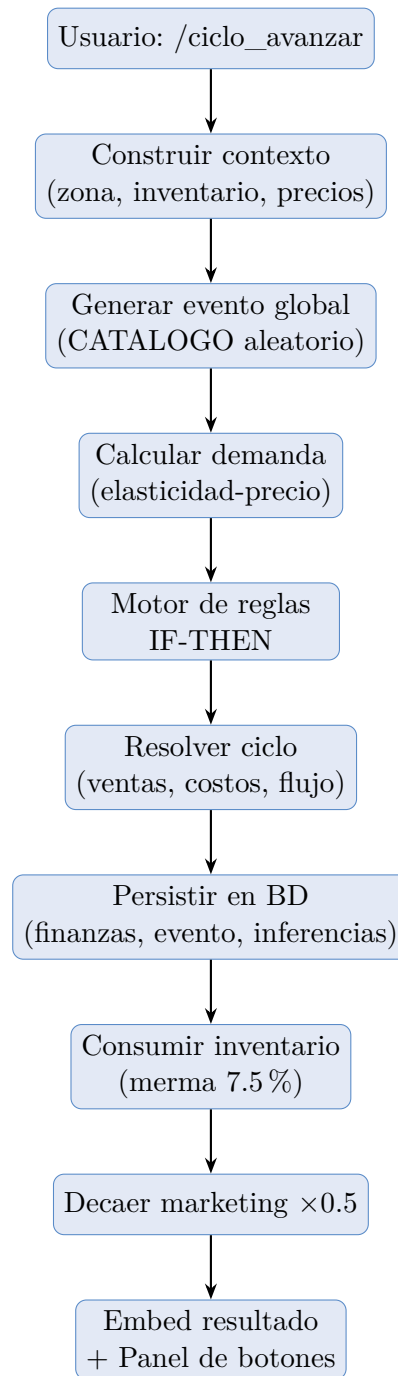
Núcleo del sistema. Ejecuta el siguiente pipeline por ciclo:

1. Construye el contexto económico del negocio (zona, inventario, marketing).
2. Inyecta el evento macroeconómico global del ciclo.
3. Calcula la demanda con elasticidad-precio.
4. Corre el motor de reglas IF-THEN.
5. Resuelve el ciclo económico (ventas, costos, flujo).
6. Persiste todo en la BD.
7. Consume inventario (con merma 7.5 %) y decae marketing.

Agente 3 — Supervisor (AgenteSupervisor)

Lee `registro_inferencias` y traduce las decisiones del motor a **lenguaje natural**. Provee la explicabilidad requerida por los sistemas expertos. El usuario lo activa con el botón **Ver explicación**.

2.3. Flujo completo de un ciclo



3. Motor de Inferencias

3.1. Arquitectura del motor

El motor evalúa una lista ordenada de funciones `Regla :: dict -> Inferencia | None`. Cada regla es **pura**: no modifica la BD, solo razona sobre el contexto. El Agente 2 ejecuta las reglas y persiste los resultados en `registro_inferencias`.

Listing 1: Estructura de una regla

```
def r_precio_alto(ctx: dict) -> Inferencia | None:
```

```

if ctx["precio_usuario"] > ctx["precio_mercado"]:
    sobreprecio = (ctx["precio_usuario"] / ctx["
        precio_mercado"] - 1) * 100
    return Inferencia(
        regla="IF precio > mercado THEN demanda cae
            exponencialmente",
        entrada=f"precio={ctx['precio_usuario']:.0f},
            mercado={ctx['precio_mercado']:.0f}",
        resultado=f"Sobreprecio de {sobreprecio:.0f}% ->
            menos clientes",
        explicacion="Cobras por encima del promedio...",
    )
return None

```

3.2. Catálogo de reglas

Regla	Condición (IF)	Acción (THEN)
r_precio_alto	precio_usuario >precio_mercado (\$55)	Dispara alerta de sobreprecio; la elasticidad castiga la demanda exponencialmente.
r_stock_insuficiente	inventario_tazas <demanda_estimada	Alerta de desabasto: se pierden ventas. Sugiere reabastecer con /proveedor.
r_grano_especialidad	tipo_grano == ."especialidad"	+15 % reputación/demanda. La calidad del grano compensa precios altos.
r_riesgo_bancarrota	caja <costos_fijos	Alerta crítica: la caja no cubre los costos fijos del ciclo.

3.3. Estructura de datos Inferencia

```

@dataclass
class Inferencia:
    regla: str # nombre legible de la regla disparada
    entrada: str # valores que la activaron
    resultado: str # que se decidio
    explicacion: str # lenguaje natural para el Supervisor
    efectos: dict # cambios a aplicar (ej. reputacion_mult
        : 1.15)

```

4. Modelo Económico

4.1. Fórmulas principales

4.1.1. Demanda con elasticidad-precio

$$D = D_{\text{base}} \cdot \left(\frac{P_{\text{mercado}}}{P_{\text{usuario}}} \right)^{1.5} \quad (1)$$

donde $P_{\text{mercado}} = \$55$ MXN/taza y el exponente 1.5 modela una elasticidad moderada-alta (especialidad en GDL es sensible al precio).

4.1.2. Unidades vendidas

$$U = \min(D, C_{\text{capacidad}}, I_{\text{inventario}}) \quad (2)$$

con $C_{\text{capacidad}} = 700$ tazas/ciclo.

4.1.3. Margen de contribución

$$MC = P_{\text{usuario}} - CV_{\text{unit}} \quad (3)$$

4.1.4. Punto de equilibrio

$$PE = \frac{CF}{MC} \quad (\text{tazas por ciclo}) \quad (4)$$

4.1.5. Flujo neto

$$FN = U \cdot P_{\text{usuario}} - (U \cdot CV_{\text{unit}} + CF) \quad (5)$$

4.1.6. Costo variable unitario con merma

$$CV_{\text{unit}} = (C_{\text{grano}} + C_{\text{leche}} + C_{\text{vaso}}) \times (1 + m) \quad (6)$$

donde $m = 7.5\%$ es la merma estimada por ciclo.

4.2. Parámetros calibrados a GDL (MXN)

Parámetro	Descripción	Valor
<i>Costos variables por taza</i>		
$C_{\text{grano comercial}}$	Grano estándar	\$6.00 MXN
$C_{\text{grano especialidad}}$	Chiapas/Veracruz/Nayarit	\$14.00 MXN
C_{leche}	Leche entera	\$4.00 MXN
C_{vegetal}	Avena/almendra	\$9.00 MXN
C_{vaso}	Empaque desechable	\$3.00 MXN
m	Merma	7.5 %
<i>Costos fijos por ciclo (mensual) — base</i>		
Renta base	$\times$ mult. zona	\$18,000 MXN
Barista Jr.	Nómina	\$8,500 MXN
Servicios (CFE/agua/gas)		\$4,500 MXN
Mantenimiento espresso		\$1,500 MXN
Total base		\$32,500 MXN
<i>Otros parámetros</i>		
P_{mercado}	Precio referencia GDL	\$55 MXN/taza
Precio inicial	Al crear negocio	\$55 MXN/taza
Caja inicial		\$100,000 MXN
Capacidad	Tazas/ciclo	700 tazas
Demanda base	$\times$ mult. zona	500 tazas

4.2.1. Ejemplo: costos variables calculados

Combinación	CV_{unit} (sin merma)	CV_{unit} (con 7.5 % merma)
Grano comercial + leche	\$13.00	\$13.98
Grano comercial + vegetal	\$18.00	\$19.35
Especialidad + leche	\$21.00	\$22.58
Especialidad + vegetal	\$26.00	\$27.95

4.3. Sistema de zonas — ZMG

La zona elegida al crear el negocio afecta la renta y la demanda base mediante multiplicadores independientes:

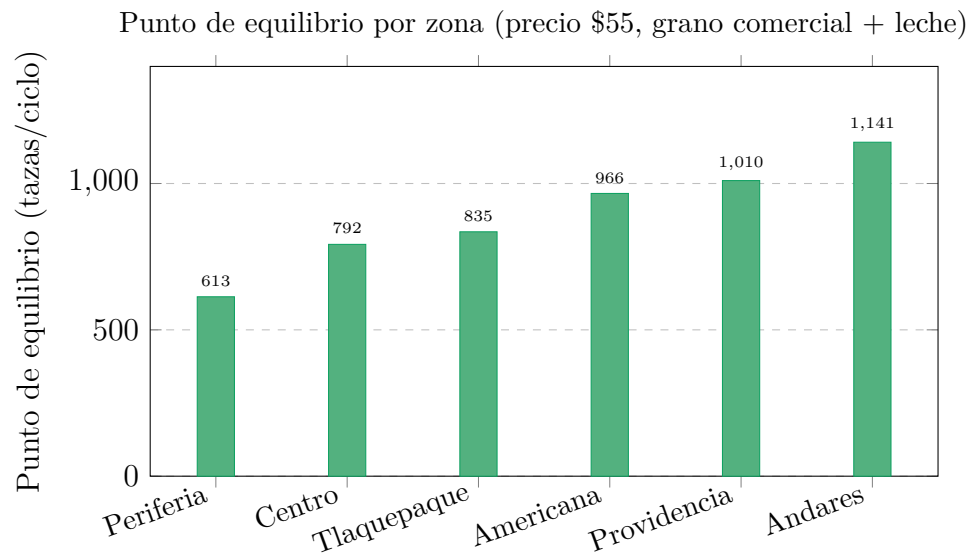
$$CF_{\text{zona}} = \$18,000 \times m_{\text{renta}} + \$14,500 \quad (7)$$

$$D_{\text{base,zona}} = 500 \times m_{\text{demanda}} \quad (8)$$

Zona	Perfil	m_{renta}	m_{demanda}	CF/ciclo
Centro Histórico	Alto flujo, turismo	1.0	1.1	\$32,500
Col. Americana / Chap.	Hub de especialidad	1.4	1.3	\$39,700
Providencia	Comercial, poder adquisitivo	1.5	1.2	\$41,500
Andares / Pta. Hierro	Premium, plazas	1.8	1.1	\$46,900
Tlaquepaque / Tonalá	Turístico, artesanal	1.1	1.0	\$34,300
Periferia	Renta baja, menos tráfico	0.6	0.8	\$25,300

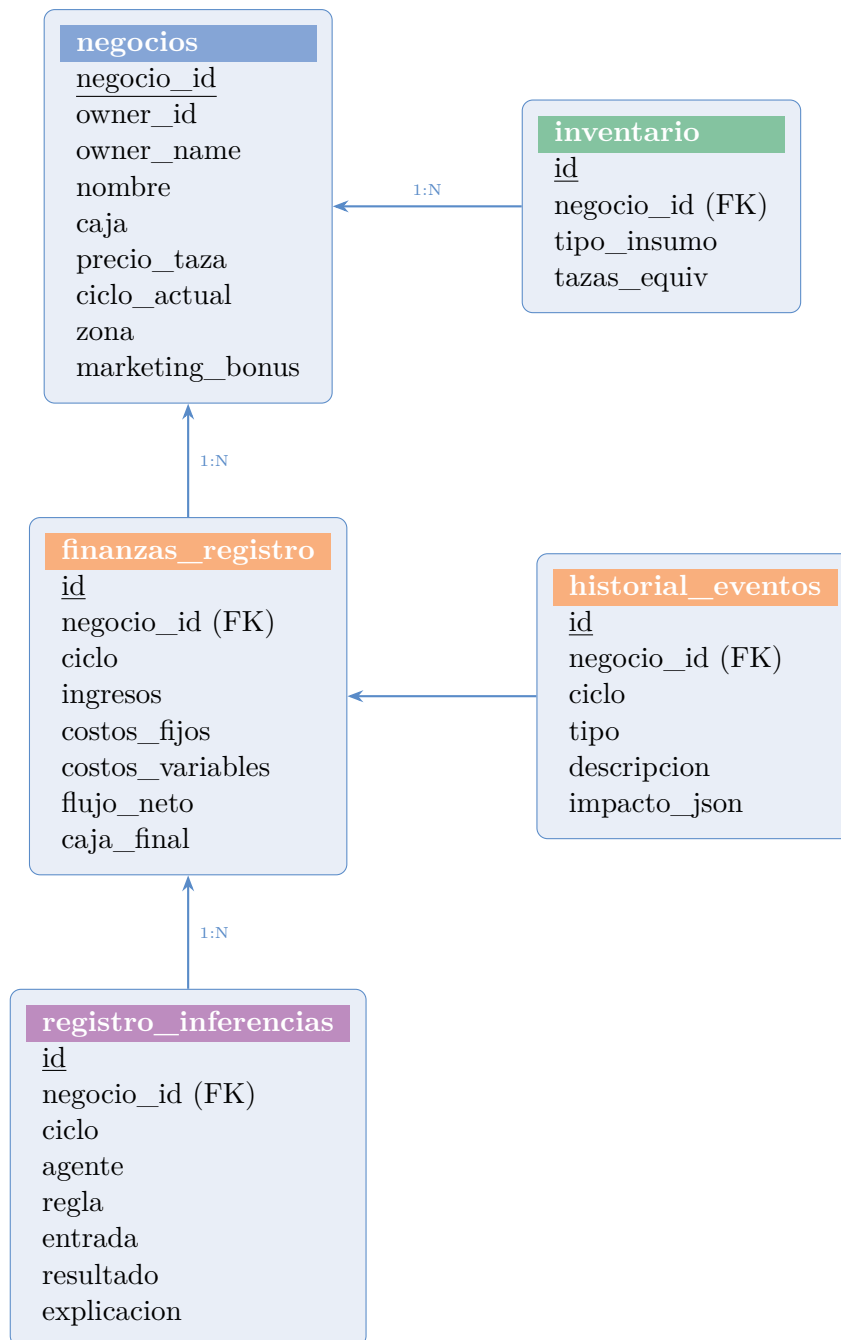
4.3.1. Análisis de rentabilidad por zona

El siguiente gráfico muestra el punto de equilibrio (en tazas/ciclo) para cada zona, asumiendo precio \$55, grano comercial + leche:



5. Base de Datos

5.1. Diagrama entidad-relación



5.2. Descripción de tablas

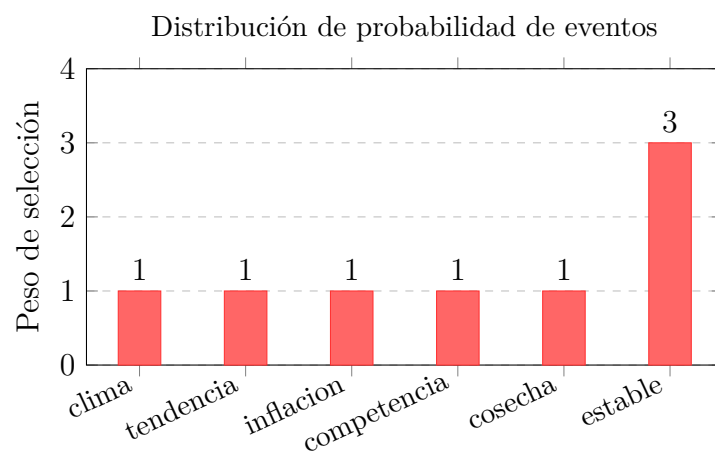
Tabla	Campo	Descripción
negocios	negocio_id	PK autoincrement
	owner_id	Discord user ID
	caja	Efectivo disponible (MXN)
	precio_taza	Precio de venta actual

Tabla	Campo	Descripción
inventario	zona	Zona ZMG elegida
	marketing_bonus	Multiplicador activo (decae 50 %/ciclo)
	tipo_insumo	grano_comercial, grano_especialidad, leche, vegetal
	tazas_equiv	Tazas que puede producir el insumo disponible
finanzas_registro	ciclo	Número de ciclo (mes)
	flujo_netto	Ingresos – (CF + CV)
	caja_final	Caja después del ciclo
historial_eventos	tipo	Categoría del evento (clima, tendencia, etc.)
	impacto_json	Multiplicadores aplicados en JSON
registro_inferencias	agente	Quién disparó la regla (pedido/atencion)
	regla	Nombre legible de la regla IF-THEN
	explicacion	Texto para el Agente Supervisor

6. Eventos Macroeconómicos

Cada ciclo el sistema elige **un evento** del catálogo mediante selección ponderada. El evento “Mes estable” tiene peso $3\times$ para que el mercado no sea caótico (peso 1 para cada evento perturbador).

Tipo	Descripción	Variable afectada	Impacto
clima	Helada en Brasil; grano sube	costo_variable_unit	$\times 1.25$
tendencia	Viral en TikTok; demanda sube	demanda_base	$\times 1.40$
inflacion	Inflación local; costos fijos suben	costos_fijos	$\times 1.05$
competencia	Rival cerca; demanda baja	demanda_base	$\times 0.85$
cosecha	Cosecha Colombia; grano baja	costo_variable_unit	$\times 0.90$
estable	Mes tranquilo	—	sin cambio



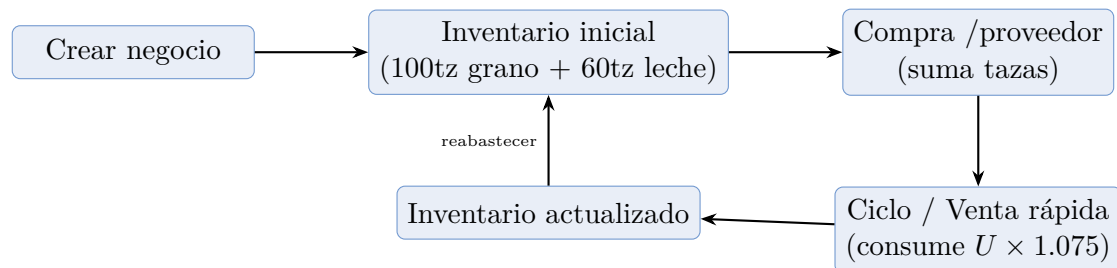
7. Sistema de Inventario

El inventario almacena insumos como **tazas-equivalente**: la cantidad de tazas que se pueden producir con el insumo disponible. Se descuenta automáticamente al vender, con una merma del 7.5 %.

7.1. Insumos disponibles

Insumo	Costo/lote	Tazas/lote	Efecto
Grano comercial	\$300 MXN	50	Costo variable base
Grano de especialidad	\$700 MXN	50	+15 % demanda, mayor CV
Leche entera	\$120 MXN	30	Insumo base
Alternativas vegetales	\$270 MXN	30	Mayor CV, segmento premium

7.2. Flujo del inventario



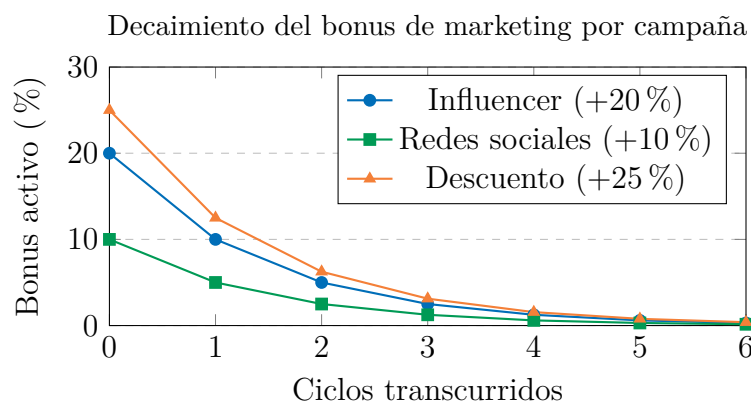
8. Sistema de Marketing

El marketing se modela como un **bonus multiplicativo** sobre la demanda base que decae un 50 % cada ciclo, incentivando la reinversión continua.

$$D_{\text{base,final}} = D_{\text{base}} \times (1 + b_{\text{mkt}}) \quad (9)$$

$$b_{\text{mkt}}^{(t+1)} = b_{\text{mkt}}^{(t)} \times 0.5 \quad (10)$$

Campaña	Costo	Bonus demanda	Contexto
Redes sociales	\$3,000 MXN	+10 %	Bajo costo, efecto moderado
Colaboración con influencer	\$8,000 MXN	+20 %	Mayor alcance
Descuento de apertura	\$1,500 MXN	+25 %	Más demanda, precio efectivo me
Evento de cata en tienda	\$5,000 MXN	+15 %	Refuerza reputación de especiali



9. Referencia de Comandos

Comando	Parámetros	Descripción
<code>/crear_negocio</code>	nombre, zona	Crea la cafetería. Elige zona de 6 opciones de la ZMG. Caja inicial: \$100,000 MXN.
<code>/ciclo_avanzar</code>	—	Simula el siguiente mes: evento, inferencias, cálculo económico, actualiza BD.
<code>/dashboard</code>	—	Panel completo: finanzas, inventario, último ciclo e indicadores. Selector si hay varios negocios.
<code>/proveedor</code>	—	Abre Select Menu para comprar insumos (descuenta caja, suma inventario).
<code>/marketing</code>	—	Abre Select Menu de 4 campañas de marketing. El bonus decae 50 %/ciclo.
<code>/ajustar_precio</code>	precio	Cambia el precio de venta. Valida que sea $\geq$ costo variable (\$13 MXN).
<code>/borrar_negocio</code>	—	Elimina el negocio y todos sus datos con confirmación doble.

9.1. Componentes interactivos (botones y selectores)

Tras ejecutar `/ciclo_avanzar` aparece el **Panel de Ciclo** con cuatro botones:

Botón	Color	Función
Avanzar ciclo	Azul (primario)	Corre el siguiente mes completo.
Ver explicación	Gris (secundario)	Muestra el reporte del Agente Supervisor.
Vender	Verde	Venta rápida sin avanzar ciclo ni aplicar costos fijos.
Pagar Nómina	Rojo	Descuenta \$8,500 MXN de caja manualmente.

10. Guía de Inicio Rápido

1. Crea tu cafetería

`/crear_negocio nombre:"La Brújula"zona:Col. Americana / Chapultepec"`

Recibirás \$100,000 MXN de caja inicial y un inventario base de 100 tazas de grano comercial y 60 de leche.

2. Consulta tu estado

`/dashboard` — revisa tu caja, inventario actual e indicadores (margen de contribución y punto de equilibrio).

3. Avanza tu primer ciclo

`/ciclo_avanzar` — el sistema generará un evento macroeconómico, calculará ventas y actualizará tu caja. Aparecerá el panel de botones.

4. Analiza las inferencias

Pulsa **Ver explicación** para que el Agente Supervisor te explique las reglas que se dispararon y por qué.

5. Reabastece y estrategiza

- `/proveedor` → compra grano de especialidad para mejorar reputación.
- `/marketing` → lanza una campaña para subir la demanda.
- `/ajustar_precio` → experimenta con el precio y observa la elasticidad.

Consejo estratégico

La zona **Col. Americana / Chapultepec** ofrece el mejor balance demanda/renta para un negocio rentable desde el ciclo 1. Las zonas premium (Andares) requieren volumen alto para superar el punto de equilibrio.

11. Ejemplo de Sesión

Listing 2: Ejemplo de salida de `/ciclo_avanzar`

```
Ciclo 3 - La Brujula
Evento global: Tendencia viral en TikTok: la demanda sube 40%.

Tazas vendidas      490
Ingresos             $26,950
Flujo neto           +$4,543
Caja final           $71,234
Reglas disparadas    1
```

Listing 3: Reporte del Supervisor (Ver explicacion)

```
Decisiones tomadas por el sistema:

- IF grano = especialidad THEN reputacion +15%
  -> El grano de especialidad mejora la percepcion de calidad
      y la reputacion de la marca.
```

12. Consideraciones Técnicas

12.1. Persistencia y transacciones

Todas las operaciones sobre la BD se realizan con `conn.commit()` explícito. Las operaciones críticas (compra de insumos, venta) verifican fondos antes de ejecutar para garantizar consistencia.

12.2. Pureza del motor de reglas

Las funciones en `rules.py` son **puras**: reciben un contexto como diccionario y devuelven una *Inferencia* o `None`, sin efectos secundarios. Esto facilita pruebas unitarias y la explicabilidad.

12.3. Escalabilidad del catálogo de reglas

Añadir una nueva regla consiste en:

1. Definir `def r_nueva(ctx: dict) -> Inferencia | None` en `rules.py`.
2. Añadirla a la lista `REGLAS`.

No se requiere modificar ningún otro módulo.

12.4. Tests sugeridos (pytest)

Listing 4: Pruebas recomendadas para la presentación

```
def test_punto_equilibrio():
    cf, cv, p = 32500, 13.98, 55.0
    pe = cf / (p - cv)
    assert abs(pe - 793) < 5

def test_elasticidad_precio():
    d = demanda_estimada(500, 55, 110) # precio doble
    assert d < 500 * 0.5 # demanda cae mas del 50%

def test_regla_bancarrota():
    ctx = {"caja": 1000, "costos_fijos": 32500, ...}
    inf = r_riesgo_bancarrota(ctx)
    assert inf is not None
```